0090. Subsets II

: Medium | Category: Backtracking | LeetCode: https://leetcode.com/problems/su

Problem

- Like Subsets, but nums may contain duplicates; the result must not contain duplicate subsets.

Intuition

- Sort so duplicates are adjacent.
- At one recursion level, picking the second "2" yields the same subtree as picking the first "2", so skip nums[i] when i > start and nums[i] == nums[i-1].

Brute Force

- Generate all subsets and dedupe with a set of sorted lists: O(n 2^n) with extra memory.

Optimal Approach

- Sort.
- Dfs(start): record path. For i from start: skip duplicates at the same level; add, dfs(i+1), remove.

Worked Example

```
[1,2,2].

[] [1] [1,2] [1,2,2]
   (skip second 2 at level 1 under [1])
[2] [2,2]
   (skip second 2 at level 0)
result 6 subsets
```

Java Solution

```java
public class SubsetsII {
    public List<List<Integer>> subsetsWithDup(int[] nums) {
        int[] a = nums.clone();
        Arrays.sort(a);
        List<List<Integer>> res = new ArrayList<>();
        dfs(a, 0, new ArrayList<>(), res);
        return res;
    }

    private void dfs(int[] a, int start, List<Integer> path, List<List<Integer>> res) {
        res.add(new ArrayList<>(path));
        for (int i = start; i < a.length; i++) {
            if (i > start && a[i] == a[i - 1]) continue;
            path.add(a[i]);
            dfs(a, i + 1, path, res);
            path.remove(path.size() - 1);
        }
    }
}
```

Complexity

- Time: O(n 2^n). Space: O(n).

Pitfalls

- The skip condition uses i > start (same level), not i > 0 (which would forbid [2,2]).